

The Rise of Hyper-Focused Software and Services (Lessons from Photoshop & Illustrator)

Introduction

In the software industry, a paradox has emerged: many of the most widely used applications are overloaded with features that most users never touch. Classic examples are Adobe Photoshop and Illustrator – industry-standard tools packed with capabilities, of which any given user might only need a small fraction. Despite their power, these monolithic applications often frustrate users and businesses with their complexity, bloated size, and high cost. This has paved the way for a new generation of **hyper-focused software companies** that aim to do *less*, but do it *better*. These companies build lean, specialized products or services tailored to specific needs, rather than one-size-fits-all behemoths. This document explores the ideas behind this shift, using Photoshop/Illustrator as a case study, and examines how focusing on the “10% of features” most users actually need can unlock business value and innovation.

The Bloatware Problem: 100% Features, 10% Usage

Modern software like Photoshop is a double-edged sword: it's incredibly feature-rich, yet that richness leads to **feature overload** and bloat. Studies show that the majority of features in typical software products are rarely or never used – one report found **80% of features in the average product are essentially waste** ¹. In fact, roughly **12% of features generate 80% of usage**, mirroring the Pareto principle ². In other words, users commonly utilize only about **5–15% of an application's capabilities** for their daily tasks. The rest is seldom touched.

This feature overload isn't just harmless extra – it has real downsides: - **Complexity and Confusion:** Extra features clutter the user interface and learning curve. For instance, newcomers to Photoshop can be overwhelmed by its countless panels and options, when they might only need basic editing tools. The *noise* of rarely-used functions can even reduce productivity for seasoned users. - **Performance and Reliability Issues:** Every additional feature adds code that must run and memory that gets used. Over years, Photoshop and similar apps have become **bloated and resource-intensive** ³. They often run slowly on modest hardware and may be prone to crashes or glitches as the codebase grows in complexity. Each new feature is another potential source of bugs. - **Security Attack Surface:** From a security standpoint, *more code = more risk*. Unused features aren't just benign – they broaden the potential attack surface of the software. As experts note, “*the more you have of [code], the more risks you run.*” ⁴ Bugs in obscure features can still be exploited by attackers. Bloated software with millions of lines of code is harder to secure than a lean, purpose-built tool. A slimmer application with only necessary features inherently has fewer vulnerabilities to worry about ⁴. - **User Frustration:** Perhaps most importantly, users often just want the core functionality to work **flawlessly**. When an application is stuffed with extraneous capabilities, core quality can suffer. A telling example is how **Adobe's flagship tools still struggle with basics like stability and version control**. It's 2026, yet Photoshop can still crash and cause users to **lose work**, with autosave/recovery not always preventing data loss ⁵ ⁶. Many users would trade the latest fancy new features for assurance that the app *never* loses their progress.

In short, software bloat means users are paying (in money, system resources, and time) for a slew of features they don't use, while often suffering subpar experience in the few features they *do* rely on. This disconnect naturally leads users and organizations to ask: *Why can't we have a tool that just does the 10% we need, really well?* Why pay for the complexity and baggage of the other 90%?

Incumbents, Lock-In, and Rent Extraction

Part of the reason this situation persists is the **market power of incumbent software vendors**. Using the Photoshop example: Adobe has dominated the creative software space for decades, establishing proprietary file formats (e.g. `.PSD` for Photoshop) and workflows that became industry standards. This created a **lock-in moat** – switching away is hard because everyone from freelancers to print shops expects you to deliver files in Adobe-compatible formats. Competing products struggle to break through that entrenched ecosystem ⁷.

With little serious competition, incumbents can get away with slowing innovation and focusing on profit extraction. Adobe's business moves illustrate this well: - **From Ownership to Subscription:** Adobe famously moved from selling perpetual licenses (buy once, use forever) to a **subscription-only model (Creative Cloud)** in the 2010s. This ensures a continuous revenue stream but forced users into *renting* software indefinitely. Many long-time customers were unhappy, feeling "locked into continuous payments to access their tools" and "*paying more without receiving more value.*" ⁸ If you stop paying, you lose access – a stark change from the old model where you could keep using your paid-for software as long as it met your needs. This **rent extraction** model benefits shareholders, but from the user perspective it can feel exploitative. As one observer noted, users now must "keep paying" even if they only use a small subset of features ⁸. - **Price Increases Over Time:** Alongside subscriptions, Adobe has imposed **frequent price hikes** on its plans ⁹. Users often see these as leveraging Adobe's near-monopoly in certain creative tools. With few alternatives that are as fully featured or industry-accepted, professionals feel they have no choice but to swallow the higher fees. This breeds resentment and the perception that the vendor is "**exploiting its market dominance**" ⁹. - **Stagnation in Core Quality:** When a vendor faces little competition, there's less incentive to fix longstanding pain points. As mentioned, crucial improvements like robust version control, crash-proof autosave, or simplifying the interface often take a backseat to flashy new features that can be marketed. Insiders argue that **lack of competition leads to complacency** – why invest heavily in overhauling fundamentals if users are locked in anyway? Indeed, Adobe has been accused of having "little incentive to innovate or offer better service" due to its monopoly-like position ¹⁰. Users who have spent years mastering Adobe software feel *trapped* in the ecosystem despite their frustrations ¹⁰.

In essence, big software companies like Adobe have been able to **maximize profit at the expense of user experience**. They bundle 100% of features and charge 100% of the price, even if a given customer only needs 10% of the functionality. This dynamic is not unique to Adobe; it appears across enterprise software and SaaS products. But the resulting discontent is fueling demand for change.

What Users Really Need: The 10% Use-Case

If most users utilize only a small slice of a product's capabilities, why not deliver just that slice as a product? Many organizations have begun asking this question. The reality is that **a simpler, focused tool can often serve customers better**: - **Usability and Productivity:** Software dedicated to the core use-case can have a cleaner UI and easier learning curve. For example, casual graphic designers might primarily need basic photo retouching and layer compositing. A Photoshop-lite that offered just those functions (and maybe a few most-wanted extras) would remove the distraction of dozens of menus and options. By tailoring the interface to the key tasks, users can work faster and with less confusion. -

Reliability and Performance: A lean tool with 1/10th the features can be optimized to run snappier and crash far less. With fewer moving parts, developers can focus on **robustness and polish** of the core functionality. It's easier to test and ensure quality on a small codebase. Users ultimately care that the feature they need works flawlessly – e.g., an artist wants their drawing app to never stutter or corrupt a file, even if it lacks 3D filters they'd never use. A focused product can pursue that robustness goal without being distracted by maintaining a giant legacy codebase. - **Desired Improvements:** Interestingly, stripping a product down to core features also creates an opportunity to add *actually desired* features that incumbents neglected. In our Photoshop example, many users would prioritize things like *better version control (history tracking, easy rollback)*, *continuous auto-save to eliminate data loss*, *cloud sync or distributed collaboration* capabilities, etc. These are features that make the **existing usage** safer and better, rather than expanding into new usage. Incumbents often overlook such improvements (which might not be “marketable” features) especially if they have technical debt that makes it hard to implement. A new purpose-built tool can include these from the start. For instance, one could design a lightweight Photoshop alternative that **never lets you lose work**, by auto-saving every change in the background and using modern cloud storage for backup – fulfilling a critical user wish that Adobe hasn't fully delivered. The hyper-focused product's pitch becomes “it does the 10% you need, and fixes the pain points you always hated in the big tool.” - **Security and Manageability:** For business and enterprise buyers, a smaller feature set also means a tighter scope to secure and manage. Many IT departments worry about the sprawling attack surface of big applications (which can include everything from scripting engines to obscure plugins). A minimalist application reduces those worries. Similarly, licensing *just* a slim tool can simplify compliance and license tracking versus managing a suite of myriad features. In regulated industries or high-security environments, having a tool that does one thing well, without extraneous capabilities, can also reduce the risk of data leakage or misuse of unneeded features.

In summary, users are increasingly looking for **tools that align with their specific workflow** – nothing more, nothing less. A common refrain is that they'd happily pay for software that covered *just their use-case* at a proportional cost, rather than paying a premium for a kitchen-sink product and using a sliver of it. This unmet need sets the stage for disruptors.

A New Opportunity: Hyper-Focused Alternatives

The gap between what incumbent software offers and what users actually use/need represents a **business opportunity**. Rather than each frustrated company trying to build an internal solution from scratch (which only a few with sufficient resources will do), new vendors can step in to create **hyper-focused products** that replace the bloated incumbents for specific segments of users. We are starting to see this trend: - **Specialized Competitors:** In the creative software space, several contenders have arisen targeting Adobe's weaknesses. For example, Serif's **Affinity Suite** (Photo, Designer, Publisher) delivers **professional-grade image editing and design tools** akin to Photoshop/Illustrator/InDesign, but with a leaner, faster experience and a much cheaper one-time purchase model ¹¹. Many users praise Affinity Photo for its intuitive interface and speed – it does most of what Photoshop's average user needs without the bulk. Another example is **Canva**, a web-based design tool that focuses on ease-of-use for common design tasks like social media graphics. Canva lacks the deep feature set of Photoshop, but for *non-professionals and small businesses* its simplicity is a feature, not a bug ¹². These products demonstrate demand for *targeted* solutions: they strip out advanced or niche capabilities to focus on the core jobs, and often can charge a fraction of the incumbent's price (or even use a freemium model). - **Success by Subtraction:** The strategy of these new companies is essentially *unbundling* the giant platforms. An incumbent like Adobe or Microsoft Office bundles dozens of functions under one umbrella. A hyper-focused challenger asks: which specific subset of those functions is underserved? Then it aims to serve that subset *better and cheaper*. We've seen this in other domains too. For instance, project management and document collaboration saw startups (Notion, Airtable, etc.) take one slice of

functionality from heavy enterprise suites and do it in a more user-friendly way. In many cases, the incumbent eventually responds or even acquires the challenger, but not before the challenger proves that users were hungry for a simplified alternative. - **Vertical or Local Niches:** An interesting aspect of the next-gen hyper-focused companies is that they can tailor to **particular industries, locales, or use-cases** even more narrowly. Instead of one product trying to please everyone worldwide, we might see (and some predict we *will* see) specialized tools *“per sector, per country, per language”*. For example, a small startup in country X could create a Photoshop-like tool optimized for that country's language and design conventions, or with features catering to the local advertising industry's common tasks. Another company might build a version of a spreadsheet app specifically for finance professionals, with built-in templates and without the bloat that generic spreadsheet software carries. These products can remain sustainable businesses at smaller scale because modern distribution (app stores, web delivery) and niche marketing let them reach their specific audience efficiently, without needing a global footprint from day one. In many cases, **domain knowledge and customer intimacy** become advantages for these focused players – they understand their narrow user base better than a giant generalist company could, and they can earn loyalty by responding quickly to those users' feedback.

For businesses evaluating software, the rise of hyper-focused alternatives is great news. It introduces **real competition** where there was little, often forcing incumbents to improve or lower prices. It also gives customers options to avoid being “locked in” to one vendor's ecosystem. If one vendor stops innovating or hikes prices, a focused competitor is likely available (or can be built) to fill that gap.

Crucially, these focused products are not about reinventing the wheel entirely – they often leverage existing technologies but package them in a right-sized solution. The barrier to creating such alternatives is coming down, thanks to new development approaches and tools, as we'll discuss next.

New Tools and AI Are Lowering the Barrier

Why is this wave of hyper-focused software happening *now*? One major reason is that it's becoming **easier and faster for small teams to build complex software** than it used to be. A combination of factors is at play: - **API Economy & Open-Source Components:** Today, a small development team doesn't need to code every feature from scratch. There are APIs and open-source libraries for almost anything – from image processing engines to UI frameworks. This means a lean product can be assembled by integrating robust existing components, focusing custom development only on the unique features or the user interface tweaks that differentiate the product. For example, an alternative image editor could use an open-source graphics library under the hood rather than reinventing that wheel. As a result, the development effort to replicate 10% of Photoshop's functionality is far less than building Photoshop was in the first place. - **AI-Assisted Development:** The advent of AI, especially large language models (LLMs) and coding assistants, has accelerated the software development process dramatically. Developers can now leverage AI tools to generate boilerplate code, suggest solutions, and even **orchestrate entire projects**. A striking recent experiment involved hundreds of AI agents collaboratively coding a functional web **browser from scratch in about one week** ¹³. While that AI-built browser (nicknamed “FastRender”) was rudimentary, it proves the concept that complex products can be created with minimal human intervention given the right orchestration. On a smaller scale, individual developers are using AI copilots (like GitHub Copilot, ChatGPT, Claude, etc.) to speed up writing code or to generate modules on demand. - **Build vs Buy – Revisited:** With these efficiencies, building a custom solution is now viable in scenarios where previously you'd automatically opt to buy off-the-shelf software. The economics have shifted. As one tech leader noted, he could build a custom call-tracking system for his small business **in 24 hours using an AI coding assistant**, replacing a third-party SaaS that would have cost hundreds per month ¹⁴. His DIY solution ended up costing only about £1 per month in cloud charges and did *“exactly what [the] business needs, nothing more, nothing less.”* ¹⁵ This example highlights that **the barrier to getting exactly what you need has dropped dramatically**

in recent years ¹⁶ . For larger organizations, this doesn't mean they'll build everything internally, but it means startups or internal teams can rapidly create alternatives when a vendor's offering becomes too costly or misaligned with their needs. The speed and cost of development have improved to the point that it's a viable competitive threat to bloated commercial software. - **Human Oversight and Product Vision:** It's important to note that human insight remains crucial. Even with AI and open-source Lego blocks, one needs a clear product vision and understanding of users to assemble a great solution. The process is not *fully* automated – rather, the new tools amplify a small team's productivity. But this means **a 5-10 person company could potentially build and maintain a product that rivals what historically took 100 people**. That shrinkage in required resources is exactly what enables hyper-focused niche products to exist profitably. They don't need millions of users to cover massive R&D costs; they can succeed on a modest user base with a lean team, thanks to development efficiency.

The takeaway is that technological advancements (AI, cloud services, open source, APIs) act as force multipliers for innovation. They reduce the incumbency advantage of sheer development manpower and big budgets. This democratizes the ability to create software, allowing more entrants to tackle tightly defined problems.

Benefits of Hyper-Focused Software for Businesses

From a business perspective – whether you're an enterprise software buyer or an entrepreneur building a startup – the hyper-focused model offers several compelling benefits:

- **Cost Efficiency:** Rather than paying for an all-in-one suite (and often overpaying relative to actual usage), businesses can **license or subscribe to smaller tools that match their needs**, often at a proportionally lower cost. For example, a company that only needs basic graphic editing might save money by using a \$50 one-time purchase alternative instead of an ongoing \$53/month Creative Cloud subscription ¹⁷ . Over time, these savings add up, and companies can redirect budget to other priorities. Additionally, if a needed feature is missing, it might even be cost-effective to sponsor its development in an open-source project or request it from a hungry young vendor, rather than accepting whatever the big vendor provides in their next release.
- **No More “Hotel Buffet” Waste:** The traditional software suite is like paying for a giant buffet when you only want a specific meal. Hyper-focused solutions let businesses **pick and choose just the functionality they require** – akin to ordering *a la carte*. This means less waste and also clearer value tracking. When you pay for a focused tool, you can directly see the ROI because it's tied to a specific workflow or outcome (e.g. “this app streamlines our image watermarking process and that alone justifies its small price”).
- **Improved Security and Compliance:** As discussed, a slimmer application surface reduces risk. Enterprises in sensitive fields (government, healthcare, finance) appreciate software that is easier to audit and secure. If a custom-focused tool has, say, 1/10th the code of a big suite and only operates within a narrow scope, it's inherently easier to test, validate, and lock down. There are fewer integration points and less mystery code. This also helps with compliance – for instance, a focused data analytics tool that does one type of analysis can be certified for compliance, whereas a giant analytics platform might have so many modules that one of them violates a requirement. **Simpler software is easier to make trustworthy.**
- **Vendor Competition = Leverage:** Having multiple smaller vendors vying to replace parts of a big suite gives customers leverage. If one vendor raises prices or falters in quality, switching to another (or even rolling out your own with help of integrators) becomes feasible. This competitive pressure keeps everyone honest. We're already seeing that organizations are **less tolerant of vendor lock-in** and are willing to piece together solutions from multiple specialized providers if it means overall better value. Software procurement can thus become more dynamic, with best-of-breed selections instead of monolithic contracts.

- **Tailored Features and Service:** Focused vendors are typically more eager to please each customer in their niche, since the **relationship is direct** and crucial for them. Customers of a niche product often enjoy more responsive support and faster turnaround on feature requests. The vendor's survival depends on keeping those core users happy (unlike a mega-corp where a single customer can feel ignored). Moreover, because the product is narrower, it's feasible for the vendor to customize or offer variants for specific large clients or local markets. For example, a French company providing an Illustrator alternative in France could ensure the UI and documentation are 100% in French, and incorporate local printing standards by default – things Adobe might never prioritize. This alignment with customer context is a win-win: users get software that *feels made for them*, and vendors can carve loyal followings.
- **Scalability of Adoption:** Interestingly, hyper-focused software can spread within organizations in a grassroots way. Employees might adopt a lightweight tool for a specific task because it's simpler, and then it grows usage internally without a top-down mandate. Because these tools usually have lower friction (cost-wise and learning-wise), adoption can scale organically. This bottom-up adoption is how alternatives like Slack (focused on team chat) displaced parts of heavier communication suites in many companies, or how Figma (focused on collaborative UI design) spread inside organizations even while Adobe XD or Photoshop were officially available. When a tool nails a use-case, users become its champions.

Of course, it's not all rosy – relying on many small tools means integration challenges and the need to manage more vendor relationships. However, modern integration (thanks to standards like REST APIs) is also easier, and companies are learning to manage modular software portfolios rather than one monolith. The benefits often outweigh the drawbacks, especially when a hyper-focused solution eliminates a pain point or a significant cost item.

Open Source and Avoiding the Next Lock-In

A critical aspect of the hyper-focused software movement is the philosophy of **avoiding the creation of new lock-in**. Many advocates argue that these new solutions should embrace open standards and open-source principles, so that customers are truly empowered and not just switching one dependency for another. Why is open source important here? - **Transparency and Trust:** Open-source software allows anyone to inspect the code. This transparency builds trust that the software does what it claims, with no hidden malicious functions, and that it can be vetted for security vulnerabilities. In sectors like government or healthcare, this is key for sovereign trust. The **Open Source Initiative** notes that openness means software can be verified to comply with local regulations and truly belong in the control of its users ¹⁸. Especially for critical infrastructure or widely used tools, being open source ensures no single vendor can unilaterally undermine the product or exploit users – the community has eyes on it. - **Community Collaboration:** Unlike proprietary development by a single company, open source invites a **community of contributors** (which may include individuals, companies, academia, government agencies – everyone) to improve the software. This model is powerful for hyper-focused tools because multiple organizations that share a need can pool their resources to build a better common solution. For instance, several graphic design studios across different countries might collectively support an open-source Illustrator alternative, each contributing plugins or translations or testing, thus creating a product none of them could build alone. This *“global commons”* approach enables smaller players to band together to take on giants ¹⁸. In Europe, for example, there is a push for open collaborative development so that the region's many small tech firms can collectively compete with the large US tech companies by not duplicating effort ¹⁹. - **No Forced Dependency:** When the code is open and data formats are standard, a customer is not beholden to a single vendor for continuity. Even if a company selling an open-source-based product goes under or changes direction, others can step in (because the code can be forked, improved, and supported by someone else). This

guards against the scenario of switching from Adobe to “Startup X” only to have Startup X hold you hostage later. If Startup X built on an open-source core, you could theoretically hire another firm to maintain that code or do it in-house if needed. Open source thus **prevents a new monopoly** from simply replacing the old one. Vendors then have to compete on service quality, customizations, and support – not on locking you in. - **Sovereign Tech and Local Control:** There’s a broader strategic angle as well, often discussed in governmental circles: *digital sovereignty*. Many countries and enterprises worry about being too dependent on foreign or third-party software providers for critical operations. Open-source, locally-supported solutions offer a path to **sovereign control**. For example, if Photoshop is critical but Adobe’s cloud-based model is undesirable, a locally developed open-source alternative gives control back to the users (and even allows self-hosting if it’s a service). We see governments funding or adopting open-source office suites, communication tools, etc., to reclaim autonomy. Open source provides the legal freedom to run and modify software without needing permission, which is essential for sovereignty. It aligns with the idea that technology critical to an organization or nation should not be exclusively under another party’s control.

To be clear, open source doesn’t automatically mean “free of cost” or that a commercial entity isn’t involved. Many hyper-focused companies may adopt an open-core model (where the core software is open source, and they sell hosted services, support, or premium extensions). The key is that the **core functionality and data formats remain open**, ensuring interoperability and longevity. This way, even as new specialized tools emerge, they collectively form an ecosystem that users can trust and invest in without fear of lock-in.

In summary, by building the next generation of software on open principles, we can have the best of both worlds: tailored solutions *and* freedom from vendor captivity. The competition then happens on who can deliver the best user experience, updates, and support for the open tool – which ultimately benefits the user most.

Conclusion: A Simpler, Smarter Software Future

Software is undergoing a paradigm shift from “do-it-all” mega applications towards **modular, focused, and user-centric tools**. The frustrations with legacy, bloated software (like our example of Photoshop/Illustrator) have illuminated a huge inefficiency – businesses spending large sums on complex products, of which they utilize only a small part. This inefficiency, combined with stagnant pain points and rising costs, created pent-up demand for alternatives. Now, fueled by modern development practices and a spirit of user-first design, a wave of hyper-focused software and service companies is rising to meet that demand.

We are likely entering an era where: - Organizations will **re-evaluate their software stacks** and trim the fat, replacing big suites with a collection of targeted solutions that together cover their needs more optimally. - Big incumbents will face real competition in areas they’ve long dominated, forcing them to improve or risk losing segments of their user base. - Niche players will thrive by staying close to their customers and delivering exactly what they need (even if it’s “only 10%” of what a giant product offered), at a fair price. - Open-source and interoperability become mainstream expectations, as buyers insist on not falling into the old traps of proprietary lock-in. - The role of AI and automation in software creation continues to grow, potentially lowering costs further and enabling an even greater diversity of software products (imagine every specific business process having its own tailor-made app, because it’s cheap to create one).

For users and businesses, this is a positive trend. It promises **more choice, more innovation, and software that aligns better with actual usage**. Instead of bending your workflow to a bloated tool’s

assumptions, you can choose tools that fit *your* workflow – or easily build one with help from AI if none exists.

The ideas described here – using only 10% of features and discarding the rest – might sound radical in a world that has long equated “more features” with “better software.” But as the industry matures, we’re learning that **simplicity and focus can drive greater productivity and satisfaction**. Just as a craftsman prefers a few perfect tools over a cluttered workbench of gizmos, software users increasingly prefer clean, dependable applications over feature-saturated suites.

The next generation of software companies will capitalize on this preference. They will be *hyper-focused*, yes – perhaps focusing on a country, an industry, or a single use-case – but collectively, they will broaden the landscape of available solutions. In doing so, they stand to reshape the software market into one that is more **competitive, user-friendly, and efficient**. The rise of hyper-focused software is essentially a return to the principle that **software should serve the user, not overwhelm them**. And that is a future worth building towards.

References (Sources)

- Pendo.io, *2019 Feature Adoption Report* – Feature usage statistics (showing ~80% of software features are rarely/never used) ¹ ² .
 - IEEE Spectrum – Analysis on software bloat and security (larger codebases increase attack surface and risk) ⁴ .
 - Adobe Community Forums – User report of Photoshop failing to auto-recover and losing work after a crash (illustrating core reliability issues) ⁵ ⁶ .
 - *Medium* (Nov 2024) – "Why Everyone is Mad at Adobe" (discussion of Creative Cloud subscription model, price increases, and software bloat frustrating users) ⁸ ³ .
 - *Medium* – Critique of Adobe's monopoly and lack of competition (users feeling locked in and Adobe lacking incentive to improve) ⁷ ¹⁰ .
 - *Medium* – Overview of alternatives to Adobe (e.g. Affinity Suite, GIMP, Canva) offering focused capabilities at lower cost or with different models ¹¹ ²⁰ .
 - Reddit (r/LLMDevs) – Report of AI agents collaboratively building a working web browser in a week, demonstrating rapid development via orchestration ¹³ .
 - LinkedIn post by Adam Carley (Oct 2025) – Case study of building a custom call-tracking tool with AI assistance in 24 hours, replacing a costly SaaS product ¹⁴ ¹⁶ .
 - Open Source Initiative (OSI) Blog – "Open Source: A global commons to enable digital sovereignty" (advantages of open source for collaboration and competition, enabling smaller companies/regions to reduce dependency on big vendors) ¹⁸ ¹⁹ .
-

1 2 The 2019 Feature Adoption Report | Pendo.io White Papers

<https://www.pendo.io/resources/the-2019-feature-adoption-report/>

3 7 8 9 10 11 12 17 20 Adobe Is Just Been A Bad Player. Why Everyone is Mad at Adobe: The Good... | by Averageguymedianow | Medium

<https://medium.com/@averageguymedianow/adobe-is-just-been-a-bad-player-bf30c3d7edcd>

4 Why Bloat Is Still Software's Biggest Vulnerability - IEEE Spectrum

<https://spectrum.ieee.org/lean-software-development>

5 6 Re: Photoshop 2024 autosave is not working - Adobe Product Community - 14123582

<https://community.adobe.com/t5/photoshop-ecosystem-discussions/photoshop-2024-autosave-is-not-working/m-p/14296360>

13 AI agents built a functional web browser in a week - impressive, but what's the real takeaway? : r/LLMDevs

https://www.reddit.com/r/LLMDevs/comments/1qfkqfw/ai_agents_built_a_functional_web_browser_in_a/

14 15 16 Building custom software faster than evaluating SaaS vendors | Adam Carley posted on the topic | LinkedIn

https://www.linkedin.com/posts/adamcarley_can-we-now-build-custom-software-faster-than-activity-7392617645113630720-Em_K

18 19 Open Source: A global commons to enable digital sovereignty - Open Source Initiative

<https://opensource.org/blog/open-source-a-global-commons-to-enable-digital-sovereignty>